The first time you use this variable in your page object ❶, WebDriver tries to find the matching HTML elements on the page. But due to the asynchronous nature of this feature, the list may not be populated yet, and your test will fail.

To get around this problem, you can configure WebDriver to not fail immediately when it can't find an element, but rather to poll the page repeatedly for a predefined period. You do this by using the `AjaxElementLocatorFactory` class when you initialize the web elements, as illustrated here:

```
public class BookingPage {
    @FindBy(css = ".typeahead li")
    private List<WebElement> typeaheadEntries;

    public BookingPage(WebDriver driver) {
      PageFactory.initElements(new AjaxElementLocatorFactory(driver,5),
                               this);
    }
    public List<String> getTypeaheadEntries() {
        List<String> entries = new ArrayList<String>();
        for(WebElement typeaheadElement : typeaheadEntries) {
            entries.add(typeaheadElement.getText());
        }
        return entries;
    }
    ...
}
```

If an element isn't found, poll the web page for up to five seconds.

Look up the type-ahead entries, with polling if necessary.

Now, if the type-ahead elements aren't ready straightaway, WebDriver will keep trying for up to five seconds before failing.

In this particular application, this approach will work just fine, but it's not foolproof. For example, suppose you have a checkout page with a `total` field that's present on the screen but initially contains no data. When the user selects an option (such as opting for travel insurance), an AJAX call will update this field. The web element corresponding to the `total` field will always be present on the page, but you need to wait for it to be populated with the correct values. In this case, the polling approach won't work, and using the `Wait` interface discussed in section 8.2.4 would be a better strategy.

8.3.2 Writing well-designed page objects

Page objects are highly reusable components, and if they're designed well, they can make your automated tests significantly easier to understand and maintain. A few simple rules can help you go a long way in designing better page objects.

PAGE OBJECTS SHOULD ONLY EXPOSE SIMPLE TYPES AND DOMAIN OBJECTS

The most important design rule for page objects is that a page object should never expose implementation details about the page or component it's encapsulating. Page object methods should accept and return simple types such as strings, dates, Booleans, domain objects, or other page objects. They should never expose WebDriver classes.

For example, the Frequent Flyer flight booking page (see figure 8.13) has many fields that could be encapsulated behind page object calls.